DIGITAL DISPLAY & COUNTING LOGIC

Curiosity Kickoff

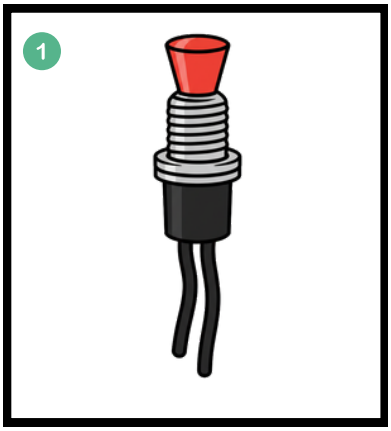
- What does a digital display show?
- How does it show numbers?
- Can a system count automatically?

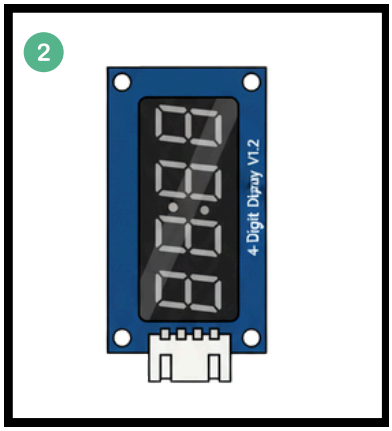
Concept Connection

A Digital Display & Counting Logic system uses electronic circuits to count events and display the results on digital displays like seven-segment LEDs. It processes input signals, increments counts, and shows numbers, making it useful in counters, timers, and automation systems.

1.push button :
A Push Button is a simple input device used to control electronic circuits. When pressed, it completes a circuit and sends a signal; when released, it breaks the connection. It is commonly used for switching, triggering actions, and user input in electronic projects.

2. 7 Segment Display :
A display made of 7 LEDs arranged in a figure-eight pattern, used to show numbers (0–9) and some letters.
Pinout: VCC (+), GND (-), DIO (Digital Input / Output) & CLK (Clock)





Class 1: Digital Display

Class Explanation

A Digital Display is an electronic device used to show numbers or information using components like seven-segment displays or LEDs. It receives signals from a circuit and visually represents data, making it useful in clocks, counters, calculators, and various electronic systems.

Materials Required

- Arduino with attached shield
- 1 × 7 Segment display
- RMC wires
- USB cable

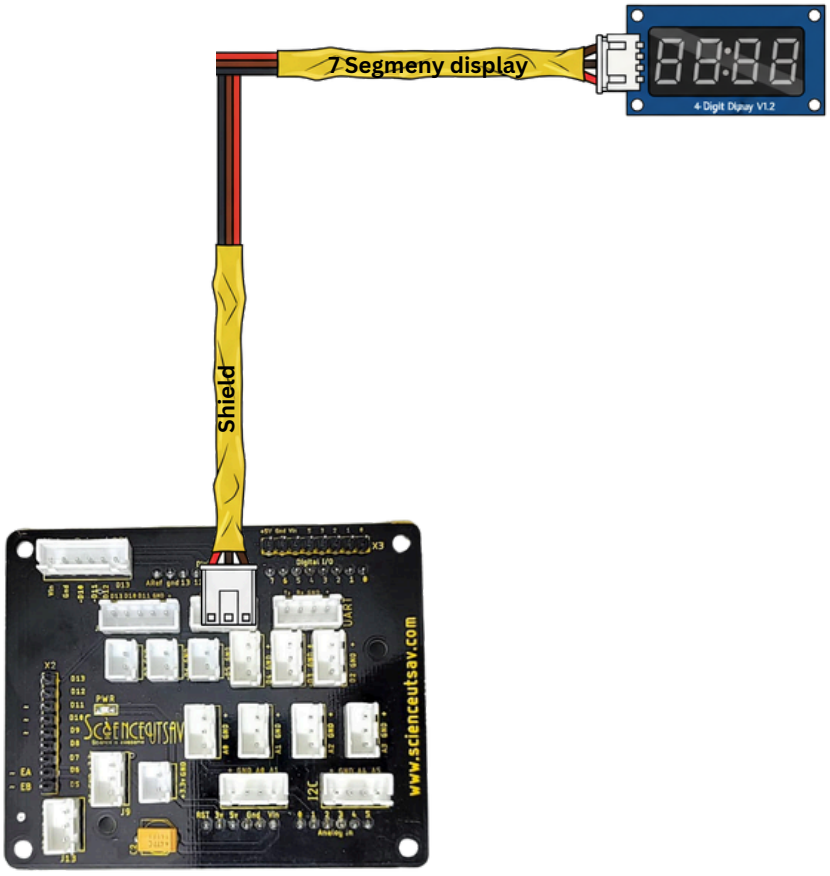
How to Connect

Use RMC wires to connect :

- 1 × 7 Segment display→ Arduino pin D8 D9 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#include <TM1637Display.h> // Include library for 4-digit 7-segment display

#define CLK 8 // Clock pin connected to D8
#define DIO 9 // Data pin connected to D9

TM1637Display display(CLK, DIO); // Create display object

void setup() {
 display.setBrightness(7); // Set display brightness (range 0 to 7)
}

void loop() {
 static int count = 0; // Start from 0 and store the count

 display.showNumberDec(count); // Show the number on display
 delay(500);          // Wait for 0.5 seconds

 count++; // Increase the number

 if (count == 10) { // Reset to 0 after 9
  count = 0;
 }
}
```

Code Explanation

- In this Arduino program, a TM1637 4-digit display is connected to pins 8 (CLK) and 9 (DIO). The TM1637Display library is used to control the display.
- The setup() function runs once when the Arduino starts. It sets the display brightness using setBrightness().
- The loop() function runs continuously. A variable count is used to store numbers starting from 0.
- The number is shown on the display using showNumberDec(), and a small delay is added so the number changes every 0.5 seconds.
- After displaying, the value of count increases. When it reaches 10, it resets back to 0.

So overall, the display continuously counts from 0 to 9 and then repeats.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Digital Display Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <TM1637Display.h>

#define CLK 8
#define DIO 9

TM1637Display display(CLK, DIO);

void setup() {
  display.setBrightness(7);
}

void loop() {
  static int count = 0;

  display.showNumberDec(count);
  delay(500);

  count = (count + 1) % 10;  // Automatically loops 0 → 9 → 0
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Digital Display Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Class 2: Digital Tap Counter

Class Explanation

A Digital Tap Counter is a system that counts the number of times a push button is pressed and displays the count on a digital display. Each press increases the count by one, making it useful for counting events, people, or objects in electronic applications.

Materials Required

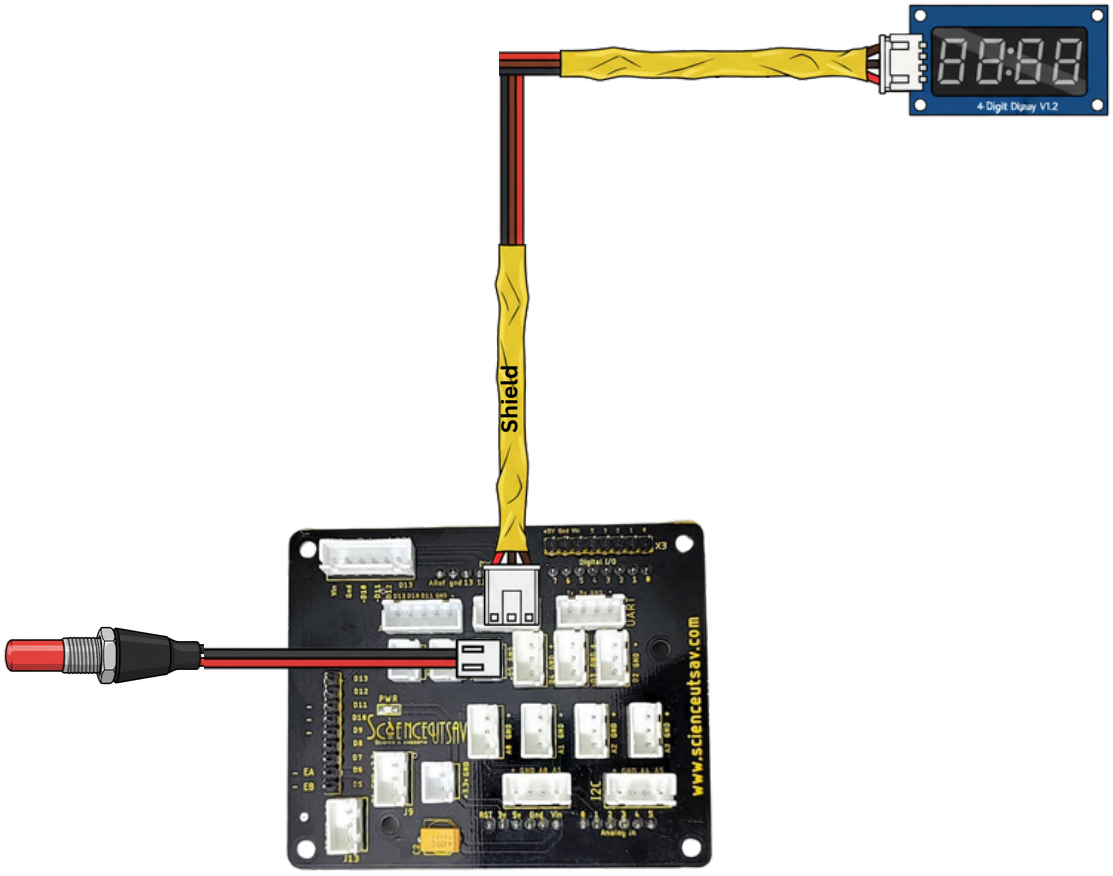
- Arduino with attached shield
- 1 × Push button
- 1 × 7 Segment display
- RMC wires
- USB cable

How to Connect

- Use RMC wires to connect :
- 1 × Push button→ Arduino pin D5 and GND
- 1 × 7 Segment display→ Arduino pin D8 D9 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#include <TM1637Display.h> // Include library for 4-digit 7-segment display

#define CLK 8 // Clock pin connected to D8
#define DIO 9 // Data pin connected to D9
#define BUTTON_PIN 5 // Push button connected to D5

TM1637Display display(CLK, DIO); // Create display object

int count = 0;     // Variable to store button press count
int lastButtonState = LOW;

void setup() {
 pinMode(BUTTON_PIN, INPUT);  // Set button pin as input
 display.setBrightness(7);   // Set display brightness (0 to 7)
 display.showNumberDec(0);   // Initialize display with 0
}

void loop() {
 int buttonState = digitalRead(BUTTON_PIN);

 // Detect button press (LOW to HIGH transition)
 if (buttonState == HIGH && lastButtonState == LOW) {
  count++;           // Increment count
  if (count > 9999) count = 0; // Reset after 9999
  display.showNumberDec(count); // Display the count
  delay(200);         // Debounce delay
 }

 lastButtonState = buttonState; // Store last button state
}
```

Code Explanation

- In this Arduino program, a push button is connected to pin 5 (BUTTON_PIN), and a TM1637 4-digit display is connected to pins 8 (CLK) and 9 (DIO). The TM1637Display library is used to control the display.

- The setup() function runs once when the Arduino starts. It sets the button pin as INPUT, sets the display brightness using setBrightness(), and shows the initial value 0 on the display.

- The loop() function runs continuously. It reads the button state using digitalRead() and compares it with the previous state stored in lastButtonState.

- When the button is pressed (LOW to HIGH transition), the count value increases by 1. The updated count is displayed using showNumberDec(). If the count exceeds 9999, it resets to 0.

- A small delay() is added to avoid multiple counts due to button bouncing.

So overall, each button press increases the count, and the value is shown on the display.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Digital Tap Counter with Arduino but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <TM1637Display.h>

#define CLK 8
#define DIO 9
#define BUTTON_PIN 5

TM1637Display display(CLK, DIO);

int count = 0;
bool running = false;   // Start/Stop flag

int lastButtonState = LOW;
unsigned long lastPressTime = 0;
int pressCount = 0;

void setup() {
  pinMode(BUTTON_PIN, INPUT);
  display.setBrightness(7);
  display.showNumberDec(0);
}

void loop() {
  int buttonState = digitalRead(BUTTON_PIN);

  // Detect button press
  if (buttonState == HIGH && lastButtonState == LOW) {
    pressCount++;
    lastPressTime = millis();
    delay(50); // debounce
  }
```

```
 // Check for double press within 400ms
 if (pressCount == 2 && (millis() - lastPressTime) < 400) {
 running = !running; // Toggle start/stop
 pressCount = 0;
 delay(200);
 }

 // Reset if too slow (not a double press)
 if ((millis() - lastPressTime) > 400) {
 pressCount = 0;
 }

 // Counting logic
 if (running) {
 display.showNumberDec(count);
 count++;
 if (count > 9999) count = 0;
 delay(500);
 }

 lastButtonState = buttonState;
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Digital Tap Counter with Arduino how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Class 3: Digital Button Counter

Class Explanation

A Digital Button Counter is a system that counts how many times a push button is pressed and displays the total on a digital display. Each press increases the count, making it useful for tracking events, operations, or quantities in electronic projects.

Materials Required

- Arduino with attached shield
- 1 × Push button
- 1 × 7 Segment display
- RMC wires
- USB cable

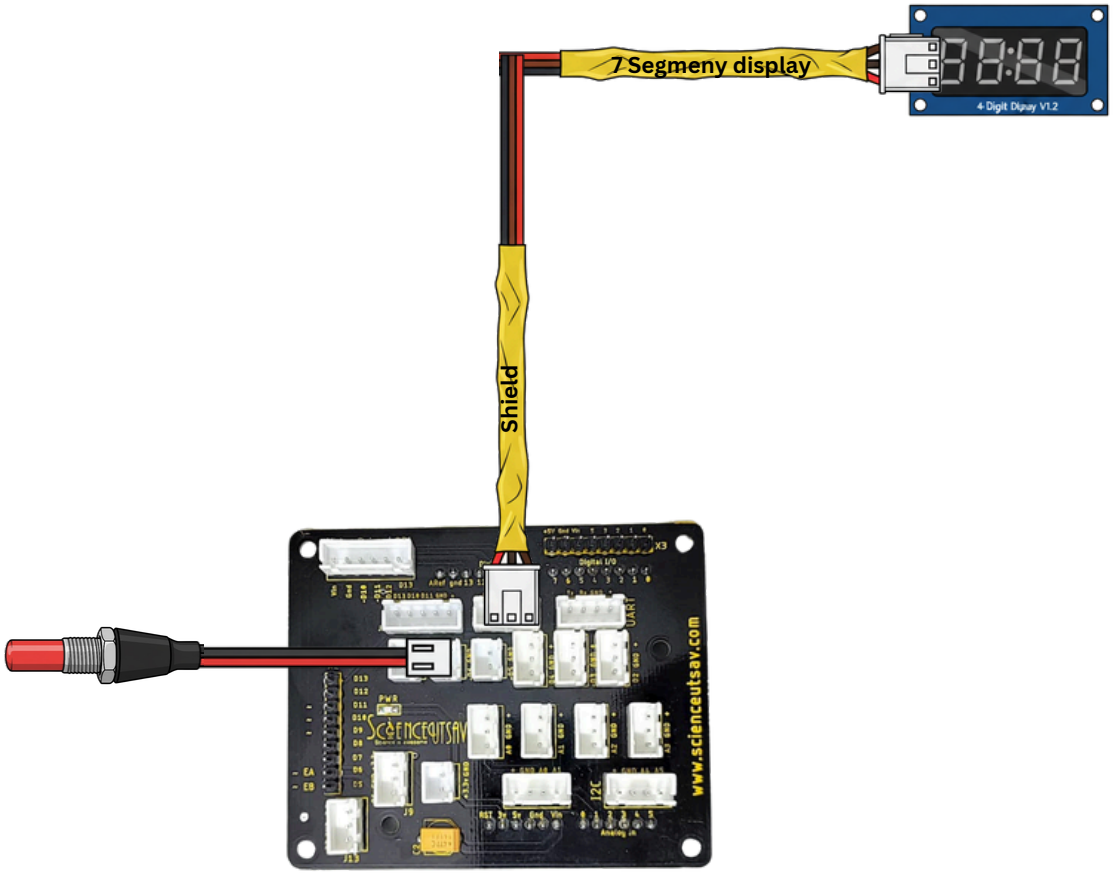
How to Connect

Use RMC wires to connect :

- 1 × Push button → Arduino pin D5 and GND
- 1 × 7 Segment display → Arduino pin D8 D9 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#include <TM1637Display.h> // Include library for 4-digit 7-segment display

#define CLK 8 // Clock pin connected to D8
#define DIO 9 // Data pin connected to D9
#define BUTTON_PIN 6 // Push button connected to D6

TM1637Display display(CLK, DIO); // Create display object

int count = 0;     // Variable to store button press count
int lastButtonState = LOW;

void setup() {
 pinMode(BUTTON_PIN, INPUT);  // Set button pin as input
 display.setBrightness(7);   // Set display brightness (0 to 7)
 display.showNumberDec(0);   // Initialize display with 0
}

void loop() {
 int buttonState = digitalRead(BUTTON_PIN);

 // Detect button press (LOW to HIGH transition)
 if (buttonState == HIGH && lastButtonState == LOW) {
  count++;           // Increment count
  if (count > 9999) count = 0; // Reset after 9999
  display.showNumberDec(count); // Display the count
  delay(200);         // Debounce delay
 }

 lastButtonState = buttonState; // Store last button state
}
```

Code Explanation

- In this Arduino program, a push button is connected to pin 5 (BUTTON_PIN) and a TM1637 4-digit display is connected to pins 8 (CLK) and 9 (DIO). The TM1637Display library is used to control the display.

- The setup() function runs once when the Arduino starts. It sets the button pin as INPUT, sets the display brightness using setBrightness(), and displays 0 initially.

- The loop() function runs continuously. It reads the current button state using digitalRead() and compares it with the previous state stored in lastButtonState.

- When the button is pressed (change from LOW to HIGH), the count value increases by 1. The updated count is shown on the display using showNumberDec(). If the count goes above 9999, it resets back to 0.

- A small delay() is used to prevent multiple counts from a single press.

So overall, each button press increases the number displayed on the screen.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Digital Button Counter but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <TM1637Display.h>

#define CLK 8
#define DIO 9
#define BUTTON_PIN 5

TM1637Display display(CLK, DIO);

int count = 0;
int lastButtonState = LOW;

void setup() {
  pinMode(BUTTON_PIN, INPUT);
  display.setBrightness(7);
  display.showNumberDec(0);
}

void loop() {
  int buttonState = digitalRead(BUTTON_PIN);

  // Detect button press
  if (buttonState == HIGH && lastButtonState == LOW) {
    count += 2;                 // Increase by 2 (even numbers)
    if (count > 9998) count = 2;  // Reset safely within 4 digits
    display.showNumberDec(count);
    delay(200);                 // Debounce
  }

  lastButtonState = buttonState;
}
```

Obective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Digital Button Counter how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Class 4: Sound Display Counter

Class Explanation

A Sound Display Counter is a system that detects sound signals using a sensor and counts each sound event. The count is then shown on a digital display, making it useful for applications like noise monitoring, event counting, or interactive electronic projects.

Materials Required

- Arduino with attached shield
- 1 × sound sensor
- 1 × 7 segment display
- RMC wires
- USB cable

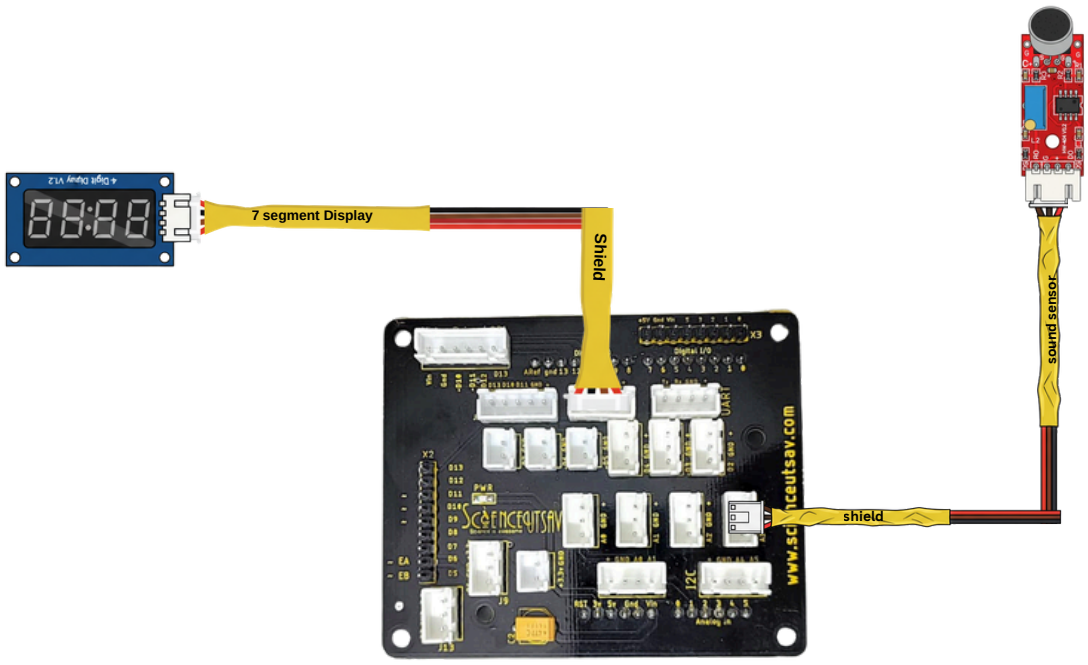
How to Connect

Use RMC wires to connect :

- 1 × Tilt sensor → Arduino pin D2 and GND
- 1 × Servo motor → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
// Sound Level → TM1637 dB Display
// Sensor: analog output on A3
// Display: TM1637 4-digit (CLK=D8, DIO=D9)
// Board: Arduino UNO (or compatible)

#include <Arduino.h>
#include <TM1637Display.h>
#include <math.h>

#define SOUND_PIN A3 // Analog output from sound sensor
#define CLK_PIN 8  // TM1637 CLK
#define DIO_PIN 9  // TM1637 DIO

TM1637Display display(CLK_PIN, DIO_PIN);

// ---- Tuning knobs ----
const unsigned long SAMPLE_MS = 50; // sampling window
const float VCC_VOLTS = 5.0;     // your board supply
const float REF_VOLTS = 0.010f;   // ~10 mV reference for dB calc
const int CAL_OFFSET = 30;     // shift reading up/down after log calc

void setup() {
 Serial.begin(9600);
 display.setBrightness(7);     // 0..7
 display.clear();
}

int readDb() {
 unsigned long start = millis();
 int signalMax = 0;
 int signalMin = 1023;

 // Capture peak-to-peak over a short window
 while (millis() - start < SAMPLE_MS) {
  int sample = analogRead(SOUND_PIN);
  if (sample > signalMax) signalMax = sample;
  if (sample < signalMin) signalMin = sample;
 }

 int p2p = signalMax - signalMin;       // peak-to-peak counts
 float volts = p2p * (VCC_VOLTS / 1023.0f);  // convert to volts
```

```
 if (volts < 0.001f) volts = 0.001f;

 float dB = 20.0f * log10f(volts / REF_VOLTS);
 dB += CAL_OFFSET;

 if (dB < 0) dB = 0;
 if (dB > 99) dB = 99;

 return (int)(dB + 0.5f);
}

void loop() {
 int dB = readDb();

 Serial.print("Approx dB: ");
 Serial.println(dB);

 display.showNumberDec(dB, true);
 delay(50);
}
```

Code Explanation

- In this Arduino program, a sound sensor is connected to analog pin A3 (SOUND_PIN) and a TM1637 4-digit display is connected to pins 8 (CLK_PIN) and 9 (DIO_PIN). The display shows the sound level in approximate decibels (dB).

- The setup() function runs once when the Arduino starts. It begins serial communication using Serial.begin(9600), sets the display brightness using setBrightness(), and clears the display.

- A separate function called readDb() is used to calculate the sound level. It reads multiple samples from the sensor for a short time and finds the maximum and minimum values. The difference (peak-to-peak value) is converted into voltage.

- Then, using a logarithmic formula, the voltage is converted into a dB value. A calibration offset is added, and the value is limited between 0 and 99.

- The loop() function runs continuously. It calls readDb() to get the sound level, prints it on the Serial Monitor, and displays it on the TM1637 display.

So overall, the Arduino measures sound intensity and shows it as a dB value on the display.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Sound Display Counter but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <Arduino.h>
#include <TM1637Display.h>

#define SOUND_PIN A3
#define CLK_PIN   8
#define DIO_PIN   9

TM1637Display display(CLK_PIN, DIO_PIN);

// Clap settings
const int CLAP_THRESHOLD = 600;   // Adjust based on your sensor
const int RESET_TIME = 3000;      // 3 seconds

int count = 0;
unsigned long lastClapTime = 0;
bool clapDetected = false;

void setup() {
  Serial.begin(9600);
  display.setBrightness(7);
  display.showNumberDec(0);
}

void loop() {
  int soundValue = analogRead(SOUND_PIN);

  Serial.println(soundValue);

  // Detect clap (sound spike)
  if (soundValue > CLAP_THRESHOLD && !clapDetected) {
    count++;                    // Increase count
    if (count > 9999) count = 0;
```

```
 display.showNumberDec(count);

 lastClapTime = millis(); // Store clap time
 clapDetected = true; // Avoid multiple triggers
 }

 // Reset clap detection when sound goes low
 if (soundValue < CLAP_THRESHOLD - 50) {
 clapDetected = false;
 }

 // Reset after 3 seconds of no clap
 if (millis() - lastClapTime > RESET_TIME) {
 count = 0;
 display.showNumberDec(count);
 }

 delay(50);
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn:
the basic idea of Sound Display Counter how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes
After completing this session, students should be able to:

- Understand how digital displays show numbers or data by receiving signals from electronic circuits in various applications.
- Learn how push button inputs are counted and displayed digitally to track events or actions accurately.
- Understand how repeated button presses are processed and counted using logic circuits and displayed on a screen.
- Learn how sound signals are detected, counted, and displayed digitally for monitoring and interactive electronic applications.

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself code
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output